In [52]:
```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor
import numpy as np
```

In [53]:
```
# 从Pytorch内置的dataset：Fasion-MINST里面下载训练数据集
training_data = datasets.FashionMNIST(
    root="E:\\pytorch\\dataset\\fasion_minst\\train_data",
    train=True,
    download=False, #这里如果设置为True，需要重新下载，时间比较慢，如果已经下载下来，则可
    transform=ToTensor(),
)
```

In [54]:
```
# 从Pytorch内置的datasetL:Fasion-MINST里面下载测试数据集
test_data = datasets.FashionMNIST(
    root="E:\\pytorch\\dataset\\fasion_minst\\test_data",
    train=False,
    download=False,
    transform=ToTensor(),
)
```

In [55]:
```
len(training_data)
```

Out[55]: 60000

In [56]:
```
len(test_data)
```

Out[56]: 10000

In [57]:
```
# 标签的字典表可以从 Github上获得
labels_dict = {
    0: "T-Shirt",
    1: "Trouser",
    2: "Pullover",
    3: "Dress",
    4: "Coat",
    5: "Sandal",
    6: "Shirt",
    7: "Sneaker",
    8: "Bag",
    9: "Ankle Boot",
}
img, label = training_data[300]

import matplotlib.pyplot as plt
figure = plt.figure(figsize=(1, 1))
plt.title(labels_dict[label])
plt.axis("off")
plt.imshow(img.squeeze(), cmap="gray")
plt.show()
```

Sandal

In [58]:
```
# 从Pytorch内置的dataset：Fasion-MINST里面下载训练数据集
training_data2 = datasets.FashionMNIST(
    root="E:\\pytorch\\dataset\\fasion_minst\\train_data",
    train=True,
    download=False,
    #transform=ToTensor(),
)
```

In [59]:
```
print(training_data2[300])
```

```
(<PIL.Image.Image image mode=L size=28x28 at 0x2282CDA9DF0>, 5)
```

In [60]:
```
print(training_data[300])
```

```
(tensor([[[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039, 0.0039,
          0.0000, 0.0000, 0.1490, 0.0902, 0.0078, 0.0000, 0.0000, 0.2863,
          0.4078, 0.0000, 0.0157, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0078,
          0.0000, 0.0000, 0.9569, 0.8431, 0.6941, 0.7216, 0.8196, 0.7059,
          0.6157, 0.0000, 0.0275, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0078, 0.0000,
          0.0000, 0.2314, 0.3059, 0.1059, 0.2157, 0.5255, 0.5137, 0.0000,
          0.0000, 0.0000, 0.0039, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.2275, 0.3216, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0039, 0.0039, 0.0078, 0.0000, 0.0039, 0.1882, 0.2431,
          0.4902, 0.2000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0314, 0.4510, 0.0588, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0078,
          0.0078, 0.0157, 0.0196, 0.0000, 0.0000, 0.5333, 0.4314, 0.1961,
          0.5255, 0.6745, 0.6745, 0.6784, 0.1255, 0.6824, 0.0627, 0.1804,
          0.8078, 0.8431, 0.1294, 0.0000],
```

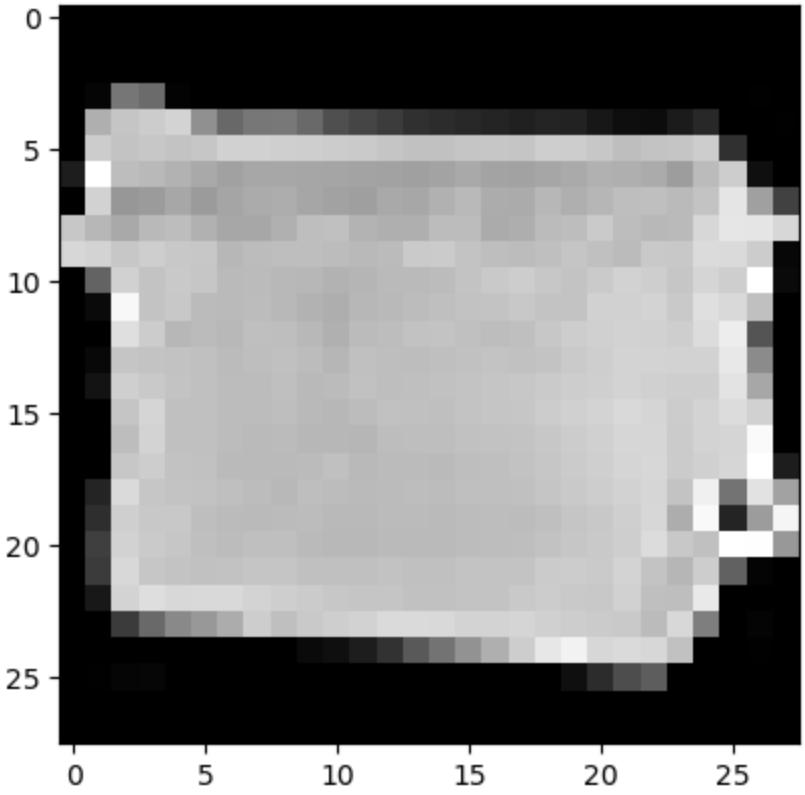
```
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0157, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.4039, 0.2706, 0.0000, 0.0000,
 0.0000, 0.0000, 0.3333, 0.7569, 0.6196, 0.7451, 0.5020, 0.8314,
 0.8000, 0.1098, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0118, 0.0000, 0.0078,
 0.1412, 0.0549, 0.1451, 0.3725, 0.0824, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.2039, 0.6118, 0.8000, 0.8745,
 0.8588, 0.0000, 0.0000, 0.0000],
[0.0078, 0.0118, 0.0078, 0.0039, 0.0000, 0.0118, 0.2353, 0.4667,
 0.4784, 0.3647, 0.2863, 0.0000, 0.0000, 0.0118, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.4196, 0.9686, 0.3137,
 0.0000, 0.0000, 0.0000, 0.0000],
[0.0118, 0.0000, 0.0000, 0.0000, 0.0000, 0.5020, 0.6510, 0.3686,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.1255, 0.8196, 0.6824, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0275, 0.0314, 0.0000, 0.6078, 0.7137, 0.7804,
 0.1882, 0.0000, 0.0706, 0.1020, 0.1725, 0.2235, 0.2745, 0.3569,
 0.4510, 0.4980, 0.5451, 0.5216, 0.8392, 0.8353, 0.7333, 0.5216,
 0.5529, 0.6078, 0.4863, 0.1882],
[0.2118, 0.5569, 0.5922, 0.6000, 0.5451, 0.6275, 0.7961, 0.7490,
 1.0000, 0.7882, 0.7647, 0.7255, 0.7059, 0.6588, 0.6000, 0.5216,
 0.4196, 0.3765, 0.3490, 0.2784, 0.3098, 0.3137, 0.2510, 0.2000,
 0.2392, 0.2667, 0.4118, 0.3020],
[0.0000, 0.0196, 0.0000, 0.0353, 0.0549, 0.0000, 0.0353, 0.0000,
 0.0000, 0.0039, 0.0039, 0.0039, 0.0039, 0.0196, 0.0431, 0.0510,
 0.0627, 0.0745, 0.0902, 0.0784, 0.0706, 0.0353, 0.3490, 0.4157,
 0.3569, 0.3608, 0.4078, 0.0549],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
 0.0000, 0.0000, 0.0000, 0.0000]]]), 5)
```

In [61]:
```
# 使用DataLoader模块进行 minibatch的数据装载，能够进行多EPOCH数据训练，且每轮训练后数据会
from torch.utils.data import DataLoader

train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)
test_dataloader = DataLoader(test_data, batch_size=64, shuffle=True)
```

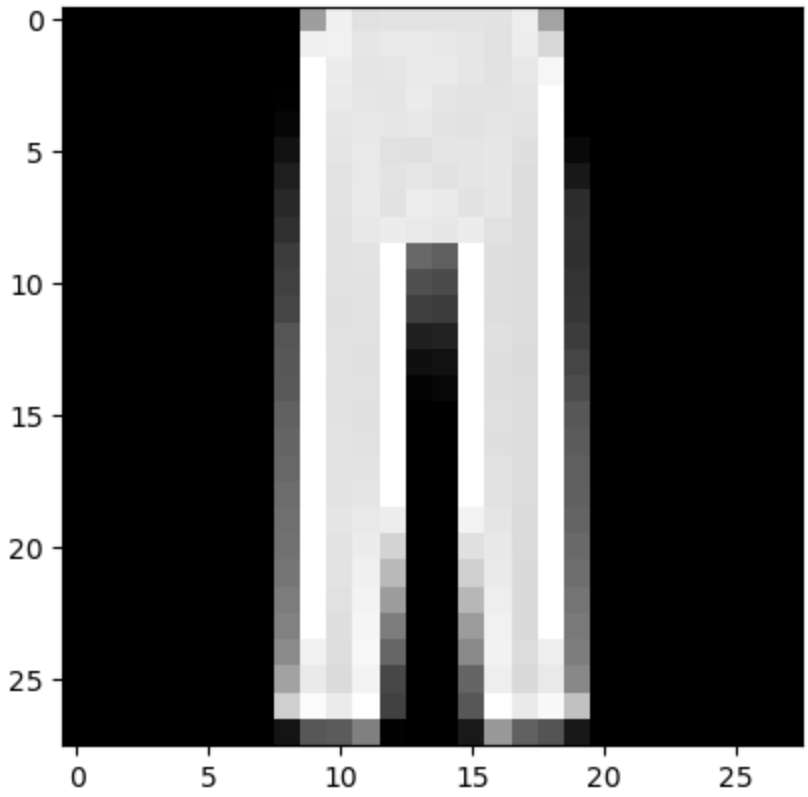
In [62]:
```
# 遍历和展示训练集的特征和对应的标签
train_features, train_labels = next(iter(train_dataloader))
print(f"Feature batch shape: {train_features.size()}")
print(f"Labels batch shape: {train_labels.size()}")
img = train_features[0].squeeze()
label = train_labels[0]
plt.imshow(img, cmap="gray")
plt.show()
print(f"Label: {label}")
```

```
Feature batch shape: torch.Size([64, 1, 28, 28])
Labels batch shape: torch.Size([64])
```



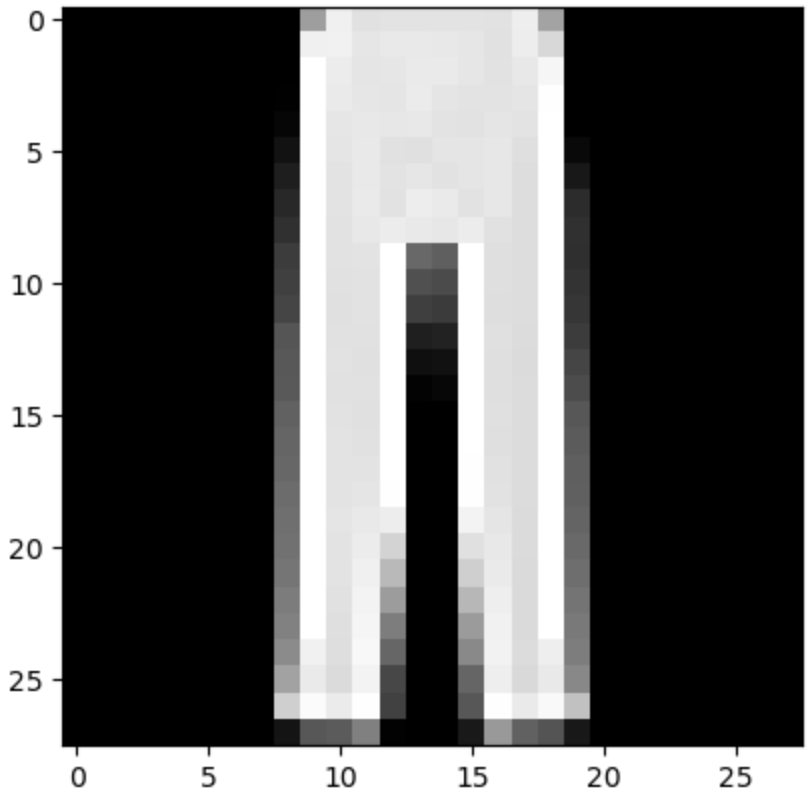

```
Label: 8
```

In [63]:
```
img = train_features[63].squeeze()
label = train_labels[63]
plt.imshow(img, cmap="gray")
plt.show()
print(f"Label: {label}")
```




```
Label: 1
```

In [64]:
```
img = train_features[63].squeeze()
label = train_labels[63]
plt.imshow(img, cmap="gray")
plt.show()
print(f"Label: {label}")
```




```
Label: 1
```

In [65]:
```python
# 指定加速设备
print(torch.cuda.is_available())
print(torch.cuda.device_count())
print(torch.cuda.get_device_name())
device = torch.cuda.current_device() if torch.cuda.is_available() else "cpu"
print(f"Using {device} device")
```

```
True
1
NVIDIA GeForce RTX 3060 Laptop GPU
Using 0 device
```

In [66]:
```python
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_avail
print(f"Using {device} device")
```

```
Using cuda device
```

In [67]:
```python
# 定义神经网络模型，继承自nn.Module,在__init__函数里面定义神经网络的分层结构;
# 在Forward()里面定义数据如何在神经网络里面传递;
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )
```

```
    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits
# 将定义好的神经网络模型发送的CUDA GPU装载
model = NeuralNetwork().to(device)
print(model)
```

```
NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
```

In [68]:
```
# 定义损失函数和优化器
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)
```

In [69]:
```
# 定义训练模型
def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)

        # Compute prediction error
        pred = model(X)
        loss = loss_fn(pred, y)

        # Backpropagation
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        if batch % 100 == 0:
            loss, current = loss.item(), (batch + 1) * len(X)
            print(f"loss: {loss:>7f}  [{current:>5d}/{size:>5d}]")
```

In [70]:
```
# 计算模型性能情况
def test(dataloader, model, loss_fn):
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    model.eval()
    test_loss, correct = 0, 0
    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(device)
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()
```

```
    test_loss /= num_batches
    correct /= size
    print(f"Test Error: \n Accuracy: {(100*correct):>0.1f}%, Avg loss: {test_loss:>
```

In [71]:

```
# 多轮迭代训练
epochs = 5
for t in range(epochs):
    print(f"Epoch {t+1}\n-------------------------------")
    train(train_dataloader, model, loss_fn, optimizer)
    test(test_dataloader, model, loss_fn)
print("Done!")
```

```
Epoch 1
-------------------------------
loss: 2.295656  [   64/60000]
loss: 2.279803  [ 6464/60000]
loss: 2.271439  [12864/60000]
loss: 2.256234  [19264/60000]
loss: 2.249012  [25664/60000]
loss: 2.239293  [32064/60000]
loss: 2.223819  [38464/60000]
loss: 2.226046  [44864/60000]
loss: 2.196453  [51264/60000]
loss: 2.155678  [57664/60000]
Test Error:
 Accuracy: 48.5%, Avg loss: 2.161916

Epoch 2
-------------------------------
loss: 2.164615  [   64/60000]
loss: 2.151326  [ 6464/60000]
loss: 2.112165  [12864/60000]
loss: 2.108495  [19264/60000]
loss: 2.069245  [25664/60000]
loss: 2.026341  [32064/60000]
loss: 2.018299  [38464/60000]
loss: 1.972205  [44864/60000]
loss: 1.957479  [51264/60000]
loss: 1.901504  [57664/60000]
Test Error:
 Accuracy: 60.7%, Avg loss: 1.902142

Epoch 3
-------------------------------
loss: 1.874133  [   64/60000]
loss: 1.875320  [ 6464/60000]
loss: 1.830256  [12864/60000]
loss: 1.725130  [19264/60000]
loss: 1.773832  [25664/60000]
loss: 1.674160  [32064/60000]
loss: 1.621894  [38464/60000]
loss: 1.639001  [44864/60000]
loss: 1.547725  [51264/60000]
loss: 1.429854  [57664/60000]
Test Error:
 Accuracy: 63.6%, Avg loss: 1.528224

Epoch 4
-------------------------------
loss: 1.538579  [   64/60000]
loss: 1.505218  [ 6464/60000]
loss: 1.283764  [12864/60000]
loss: 1.396351  [19264/60000]
loss: 1.452993  [25664/60000]
loss: 1.306817  [32064/60000]
loss: 1.281425  [38464/60000]
loss: 1.237958  [44864/60000]
loss: 1.266900  [51264/60000]
```

```
loss: 1.153420  [57664/60000]
Test Error:
 Accuracy: 64.4%, Avg loss: 1.248107

Epoch 5
-------------------------------
loss: 1.284576  [   64/60000]
loss: 1.206654  [ 6464/60000]
loss: 1.187574  [12864/60000]
loss: 1.030694  [19264/60000]
loss: 1.112178  [25664/60000]
loss: 1.168069  [32064/60000]
loss: 1.035437  [38464/60000]
loss: 1.161988  [44864/60000]
loss: 1.106181  [51264/60000]
loss: 1.160079  [57664/60000]
Test Error:
 Accuracy: 64.4%, Avg loss: 1.077487

Done!
```

In [72]:
```python
# 保存模型（持久化模型）
torch.save(model.state_dict(), "E:\\pytorch\\model_result\\fasion_MINST\\model.pth"
print("Saved PyTorch Model State to model.pth")
```

```
Saved PyTorch Model State to model.pth
```

In [73]:
```python
model = NeuralNetwork().to(device)
model.load_state_dict(torch.load("E:\\pytorch\\model_result\\fasion_MINST\\model.pt
```

Out[73]:
```
<All keys matched successfully>
```

In [74]:
```python
# 模型应用：

classes = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot",
]

model.eval()
x, y = test_data[0][0], test_data[0][1]
with torch.no_grad():
    x = x.to(device)
    pred = model(x)
    predicted, actual = classes[pred[0].argmax(0)], classes[y]
    print(f'Predicted: "{predicted}", Actual: "{actual}"')
```

```
Predicted: "Ankle boot", Actual: "Ankle boot"
```